LAB 02

System calls

gdb, trace e freemem: observar o caminho até o kernel

Aluno(a)	_____ Data: ____/____/_____

Objetivo: ver uma system call no gdb e instalar duas system calls novas (trace e freemem). Você só copia, cola e analisa.

ETAPA 1 /6 Preparar

Abra o ambiente:

```
bash xv6.sh
```

ETAPA 2 /6 Ver uma system call no gdb

Terminal 1	Terminal 2 (rode bash xv6.sh de novo)
make qemu-gdb	gdb

No gdb (terminal 2):

```
b syscall
c
bt
n
n
p p->trapframe->a7
p p->name
p p->pid
quit
```

Pergunta	Resposta
Quem chamou syscall()? (bt)	*usertrap() – kenel/trap.c:67*

Pergunta	Resposta
Valor de a7	*7*
Que syscall é? (kernel/syscall.h)	*SYS_exec*
Nome e PID do processo	*1*

Depois: Ctrl-a, x no terminal 1.

ETAPA 3 /6 Instalar a system call trace (copiar e colar)

Com o xv6 desligado, cole os 5 blocos no terminal (xv6), um de cada vez.

1) Número e declarações (syscall.h, user.h, usys.pl, defs.h)

```
grep -q SYS_trace kernel/syscall.h || cat >> kernel/syscall.h << 'EOF'
#define SYS_trace 22
#define SYS_freemem 23
EOF
grep -q 'trace(int)' user/user.h || echo 'int trace(int);' >> user/user.h
grep -q '"trace"' user/usys.pl || echo 'entry("trace");' >> user/usys.pl
grep -q traceprint kernel/defs.h ||
echo 'void traceprint(struct proc*, int);' >> kernel/defs.h
```

2) Cada processo guarda sua máscara (proc.h) e o filho herda (fork)

```
grep -q tracemask kernel/proc.h ||
sed -i '/char name\[16\];/a\ int tracemask;' kernel/proc.h
P=kernel/proc.c
grep -q tracemask $P ||
sed -i '/safestrcpy(np->name/a\ np->tracemask = p->tracemask;' $P
```

3) A system call: lê o argumento e guarda a máscara (sysproc.c)

```
grep -q sys_trace kernel/sysproc.c || cat >> kernel/sysproc.c << 'EOF'

uint64
sys_trace(void)
{
  int mask;
  argint(0, &mask); // 1o argumento (registrador a0)
  myproc()->tracemask = mask; // guarda no processo
  return 0;
}
EOF
```

4) Tabela de system calls e impressão do trace (syscall.c)

```
grep -q traceprint kernel/syscall.c || {
S=kernel/syscall.c
sed -i '/^extern uint64 sys_close/a extern uint64 sys_trace(void);' $S
sed -i '/^\[SYS_close\]/a [SYS_trace] sys_trace,' $S
sed -i '/= syscalls\[num\]();/a\ traceprint(p, num);' $S
cat >> $S << 'EOF'

static char *nomes[] = {
[SYS_fork] "fork", [SYS_exit] "exit", [SYS_wait] "wait",
[SYS_pipe] "pipe", [SYS_read] "read", [SYS_kill] "kill",
[SYS_exec] "exec", [SYS_fstat] "fstat", [SYS_chdir] "chdir",
[SYS_dup] "dup", [SYS_getpid] "getpid", [SYS_sbrk] "sbrk",
[SYS_sleep] "sleep", [SYS_uptime] "uptime", [SYS_open] "open",
[SYS_write] "write", [SYS_mknod] "mknod", [SYS_unlink] "unlink",
[SYS_link] "link", [SYS_mkdir] "mkdir", [SYS_close] "close",
[SYS_trace] "trace", [SYS_freemem] "freemem",
};

// chamada em syscall(): imprime se o bit num da mascara estiver ligado
void
traceprint(struct proc *p, int num)
{
  if((p->tracemask >> num) & 1)
    printf("%d: syscall %s -> %d\n",
           p->pid, nomes[num], (int)p->trapframe->a0);
}
EOF
}
```

5) Registrar o programa trace (user/trace.c já existe)

```
grep -q _trace Makefile || sed -i '/_zombie/a\\t$U/_trace\\' Makefile
```

Veja o que mudou em syscall.c:

```
git diff kernel/syscall.c
```

✍ **SUAS OBSERVAÇÕES**

a) Em syscall(), em que registrador o kernel lê o número da system call? Onde guarda o retorno?
b) Por que o fork precisa copiar o tracemask para o filho?

a) O kenel lê o número da system call no registrador *a7* e armazena o resultado no registrador *a0*. *(num = p->trapframe->a7; p->trapframe->a0 = syscalls[num]();)*.

b) Porque o processo filho precisa herdar a configuração de rastreamento do pai. Assim, as system calls realizadas pelo filho também podem ser monitoradas conforme a máscara definida, mantendo o rastreamento após o *fork()*.

ETAPA 4 /6 Testar o trace

```
make qemu
```

Comando no xv6	Quantas linhas de trace?
`trace 32 grep hello README`	*4*
`trace 65536 echo ola SO`	*4*
`grep hello README`	*0*

Máscara	Conta	Qual syscall? (kernel/syscall.h)
32	1 << 5	*read*
65536	1 << 16	*write*

> ✍ **SUAS OBSERVAÇÕES**
>
> No trace 65536 echo ola SO, por que o texto e as linhas do trace saem misturados?
>
> Porque o *echo* escreve o texto no console por meio do system call *write* e o kernel imprimi uma mensagem de rastreamento após cada chamada. Como ambos utilizam o mesmo console, as mensagens aperecem intercaladas.

ETAPA 5 /6 Instalar a system call freemem (copiar e colar)

Desligue o xv6 (Ctrl-a, x) e cole os 3 blocos:

1) Kernel: contar as páginas livres (kalloc.c e sysproc.c)

```
grep -q kfreemem kernel/kalloc.c || cat >> kernel/kalloc.c << 'EOF'

// conta as paginas livres percorrendo a lista de paginas livres
uint64
kfreemem(void)
{
  struct run *r;
  uint64 n = 0;
  acquire(&kmem.lock);
  for(r = kmem.freelist; r; r = r->next)
    n++;
  release(&kmem.lock);
```

```
  return n * PGSIZE; // paginas -> bytes
}
EOF
grep -q kfreemem kernel/defs.h ||
echo 'uint64 kfreemem(void);' >> kernel/defs.h
grep -q sys_freemem kernel/sysproc.c || cat >> kernel/sysproc.c << 'EOF'

uint64
sys_freemem(void)
{
  return kfreemem();
}
EOF
```

2) Ligar a system call (user.h, usys.pl, syscall.c)

```
grep -q freemem user/user.h || echo 'int freemem(void);' >> user/user.h
grep -q freemem user/usys.pl || echo 'entry("freemem");' >> user/usys.pl
grep -q sys_freemem kernel/syscall.c || {
S=kernel/syscall.c
sed -i '/^extern uint64 sys_close/a extern uint64 sys_freemem(void);' $S
sed -i '/^\[SYS_close\]/a [SYS_freemem] sys_freemem,' $S
}
```

3) Programa memlivre

```
cat > user/memlivre.c << 'EOF'
#include "kernel/types.h"
#include "user/user.h"

#define PAG 4096

int
main(int argc, char *argv[])
{
  int m0 = freemem();
  printf("[1] inicio: %d pags livres\n", m0/PAG);

  sbrk(10 * PAG); // pede 10 paginas a mais
  int m1 = freemem();
  printf("[2] sbrk: %d livres (-%d)\n", m1/PAG, (m0-m1)/PAG);

  int pid = fork();
  if(pid == 0){
    int m2 = freemem();
    printf("[3] filho: %d livres (-%d)\n", m2/PAG, (m1-m2)/PAG);
    exit(0);
  }
  wait(0);
  int m3 = freemem();
  printf("[4] fim: %d livres (-%d desde [2])\n", m3/PAG, (m1-m3)/PAG);
  exit(0);
}
EOF
```

```
grep -q _memlivre Makefile || sed -i '/_zombie/a\\t$U/_memlivre\\' Makefile
```

```
make qemu
$ memlivre
```

Linha	Páginas livres	Variação
[1] inicio	*32529*	-
[2] sbrk	*32519*	*10*
[3] filho	*32498*	*21*
[4] fim	*32519*	*0 em relação a [2]*

✍ **SUAS OBSERVAÇÕES**

O fork gastou mais páginas do que o processo tem. O que mais o kernel cria para o filho?

Além de copiar a memória do processo pai, o kernel precisa criar estruturas para gerenciar o filho, como a tabela de páginas, a pilha do kernel, o *trapframe* e outras estuturas de controle do processo. Por isso, o *fork()* consome mais memória do que apenas as páginas utilizadas pelo programa.

ETAPA 6 /6 Teste automático

Com o xv6 desligado (Ctrl-a, x):

```
./grade-lab-syscall trace
```

Teste	OK / FAIL
trace	*Ok*